

AutoML and AutoRAG Strategies

Version 1, 2026-09-24

Home

Welcome

Across the industry, organizations are learning a hard truth: getting an AI data pipeline reliably into production is still remarkably difficult. More than 75% of enterprise AI data initiatives stall out, trapped inside exploratory notebooks that work on a developer's laptop but never survive the transition to a governed, scalable production endpoint.

Rather than focusing on platform administration or YAML cluster setup, this course places you directly in the practitioner's seat (Data Scientists, AI Engineers, and Technical Solution Architects).

You will explore how Red Hat OpenShift AI's "Auto" capabilities: AutoML, AutoRAG, and MLflow governance; take raw enterprise data sitting in object storage and systematically convert it into production-ready, auditable inference APIs in hours rather than months.

Goal

Teach technical professionals how to strategically evaluate use cases, and apply AutoML (predictive modeling) and AutoRAG (generative retrieval) in Red Hat OpenShift AI 3.5+, to drive measurable business value by showcasing its targeted automated ML/RAG capabilities.



In OpenShift AI 3.5, AutoML and AutoRAG are **technology preview** features. Both are targeted to reach **general availability** in 3.6. This course teaches the 3.5 workflow and flags what changes at GA.

Objectives

On completing this course, you should be able to:

- **Predictive Foundations (AutoML):** Evaluate when to apply AutoML to convert raw tabular data or time series into production-ready predictive models across classification, regression, and time-series tasks.
- **AutoML governance & deployment:** Use AutoML evaluation artifacts (leaderboards, feature importance, ROC/PR curves) to validate business logic and promote winning models to production.
- **GenAI prompting & MLflow governance:** Transition from predictive data to generative text, leveraging the GenAI Playground and MLflow to version, track, and eliminate "prompt drift".
- **Generative retrieval (AutoRAG):** Position AutoRAG as an automated optimization engine that benchmarks chunking, embedding, and retrieval configurations (search mode, top-k, and more) to build grounded LLM applications.
- **Knowledge review and assessment:** Track core enablement concepts across AutoML, MLflow prompt tracking, and AutoRAG to validate practitioner capability in diagnosing enterprise data scenarios and selecting optimal AI patterns.

Audience

- **Primary Personas:** Data Scientists, AI Engineers, and Technical Solution Architects evaluating or adopting Red Hat OpenShift AI 3.5+.
- **Technical Level & Focus (TL2):** Technical practitioners and field specialists looking to understand the "why", decision points, trade-offs, and business outcomes of automated ML/RAG capabilities. Focused on transitioning from manual ML/RAG development without needing to perform cluster admin setup procedures.
- **Distribution & Accessibility:** Publicly discoverable enablement designed for global Red Hat field teams, ecosystem partners, and enterprise customers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with core machine learning concepts (classification, regression, time series).
- Familiarity with generative AI concepts (LLMs, prompting, retrieval-augmented generation).
- Familiarity with Red Hat OpenShift AI 3.5+ from a practitioner (Data Scientist / AI Engineer) perspective.
- No cluster administration or infrastructure setup experience is assumed or required.

Classroom environment

This course requires no hands-on user labs. It uses Summit or created Arcade interactive clickthroughs, and video walkthroughs to demonstrate AutoML and AutoRAG capabilities, and it closes with a scenario-based graded knowledge check.

Other sources of information about Red Hat OpenShift AI

For documentation about Red Hat OpenShift AI, see either its [product documentation](#) or the [Red Hat AI Examples](#) upstream repositories.

The Production Team

Aditi Saluja

Suhas Kashyap

Karlos Knox

Anna Swicegood



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Predictive Foundations with AutoML

Goal

Evaluate when to use AutoML to convert raw tabular data or time series into production-ready predictive models across classification, regression, and time-series tasks.

AutoML in Red Hat OpenShift AI 3.5+ takes the raw tabular data sitting in your S3 bucket, automates the tedious middle (data prep, feature engineering, algorithm selection, and ensembling), and hands you back a ranked leaderboard of production-ready candidate models. Not in months, but in hours.

This chapter is about the **foundations**: why this matters to the business, where predictive AI fits alongside the LLMs everyone's talking about, how to tell whether your data is even a good fit. How to match any business problem to the right kind of prediction. By the end you won't be running models yet, that comes later, but you'll know exactly when AutoML is the right tool to reach for, and why.

Why AutoML? The Business Case

Objective

Explain, in language a business leader will care about, what AutoML actually automates and the return it delivers.

Let's start with the part that gets budget approved, because if you can't explain the "why" to the person signing the checks, the coolest model in the world never ships.

The problem AutoML is solving

The data scientist does heroic work, cleaning messy columns, inventing features, testing XGBoost against something they read about last week, tuning knobs until the accuracy nudges up another half a percent. It's skilled, careful, **slow** work. And a huge amount of it is the same work, on every project.

That repetition is exactly what AutoML automates. Instead of a person hand-running each step, the platform handles:

- **Data preparation**, inferring column types, imputing missing values, encoding categories.
- **Feature engineering**, turning raw columns into signals a model can learn from.
- **Algorithm selection**, trying many model families in parallel instead of one at a time.
- **Ensembling**, combining the best candidates into something stronger than any single model.

We bring the judgment, the business question, the data, the decision about what "good" looks like. AutoML takes over the grind in the middle.

The "sorting specialist"

If you want a mental picture, think of AutoML as a **Sorting Specialist**. You feed in raw tabular CSV data pulled from S3 storage, the platform reads the patterns hiding in it, and out comes a **ranked leaderboard** of candidate models, sorted best to worst, ready for you to inspect and promote. You're no longer building models one at a time and hoping. You're reviewing a graded shortlist.

Interactive experience: click the flashing purple circle to advance the frame.

<iframe

```
src="https://demo.arcade.software/HhxRKh4UPrF9NNfV9oVA?embed&embed_mobile=inline&embed_desktop=inline&show_copy_link=true"
width="100%"
height="600px"
frameborder="0"
allowfullscreen
webkitallowfullscreen
mozallowfullscreen
allow="clipboard-write"
muted>
</iframe>
```

The return on investment

Here's how that translates when it's your turn to justify the spend:

- **70–80% faster time-to-market.** Development cycles that used to run three to six months collapse into a matter of hours. Multi-quarter R&D efforts become same-week operational delivery.
- **A 3x workforce multiplier.** Software engineers, data analysts, and IT practitioners can stand up high-accuracy predictive baselines themselves, which frees our specialists for the genuinely hard problems.
- **Path to production.** This is the quiet one that matters most. Because AutoML produces **standardized deployment artifacts**, the winning model moves directly toward a production endpoint instead of getting stranded in a notebook.

What's next

Next we'll place predictive AI next to the large language models everyone's talking about, so we can see where each one earns its keep, and learn how to tell whether our data is even a candidate for AutoML.

Where Predictive AI Fits (and Where It Doesn't)

Objective

Position predictive models against LLMs on cost and behavior, and qualify whether a dataset is a

good fit for AutoML.

Walk into almost any room in 2026 and the conversation is about large language models. Which is great, LLMs are undisputedly transformational. But there’s a trap hiding in all that excitement: organizations are reaching for a giant GPU-hungry model to answer a question that a tiny, boring, deterministic model would answer better, cheaper, and a thousand times faster.

Predictive models vs. LLMs

The easiest way to see it is side by side.

Dimension	Predictive Models (AutoML / Tabular)	Large Language Models (LLMs)
Primary scope	Narrow, single-task structured decisions (churn, fraud, pricing)	Broad, open-domain reasoning and text generation
Input structure	Low-dimensional structured tabular data (10–100+ features)	High-dimensional unstructured text across billions of tokens
Artifact size	Kilobytes to megabytes (lightweight trees / ensembles)	Tens to hundreds of gigabytes (billions of parameters)
Hardware	Lightweight CPU infrastructure, no specialized GPUs	High-VRAM GPU clusters
Latency	Microsecond inference for high-throughput systems	Second-level response times
Auditability	Deterministic logic with clear feature importance	Probabilistic reasoning needing guardrail evaluation

Predictive models are cheap, fast, and deterministic. They scrape millions of rows on the CPU clusters you already own, at a fraction of the cost of a GPU fleet, and they give you the same answer every time with a clear paper trail for **why**.

Is your data even a candidate?

AutoML is powerful, but it’s not magic, it expects a certain shape of data. Before you kick off a run, check that your dataset qualifies:

- **Tabular structure.** Rows are entities (records), columns are attributes (properties), stored as CSV. Files should be UTF-8, comma-delimited, with a header row. You can upload up to 32 MiB through the dashboard UI, or ingest up to 100 MB directly from S3.
- **A clear target label.** One column holds the ground-truth answer you want to predict for new data.
- **Feature diversity.** A healthy mix of numeric and categorical columns.
- **Enough volume.** At least 500–1,000 rows to prototype, and 10,000+ is typical for a serious enterprise evaluation.

When AutoML is not the tool

Just as important is knowing when to put it down and reach for something else:

- **Unstructured media:** images, audio, video. Those need specialized deep-learning pipelines.
- **Open-ended text generation:** chatbots, document synthesis, summarization. That's the world of LLMs and AutoRAG, which we'll get to later in the course.
- **Custom domain edge cases:** problems that hinge on hand-crafted business rules a model can't infer from the raw table.

Match the tool to the shape of the problem and AutoML feels like a superpower. Force it onto the wrong problem and it just frustrates everyone.

What's next

Next we'll determine how to match predictive business problems to one of the four AutoML task types.

The Four Prediction Task Types

Objective

Match any predictive business problem to one of the four AutoML task types and its default optimization metric.

Here's the good news: no matter how complicated the business problem sounds, every predictive question you'll ever hand to AutoML boils down to one of just **four** task types. Learn to spot which one you're looking at and you're most of the way there.

Even better, you often don't have to decide alone. When you load a CSV into the Red Hat OpenShift AI dashboard, it inspects your columns, infers their types, and **recommends** a task type and a sensible default metric. You should understand the four tasks well enough to know when it's right.

Start by reading your data, not the problem statement

If this is your first time driving one of these tools, here's the habit to build early: don't start from how the business **describes** the problem, start from what your data actually looks like. The stakeholder might say "I want to know who's going to leave us," but the platform doesn't see intentions, it sees columns. So the very first thing you do is open your CSV and look hard at **one** column: the **target**, the value you want the model to predict for new rows.

Two quick questions answer almost everything:

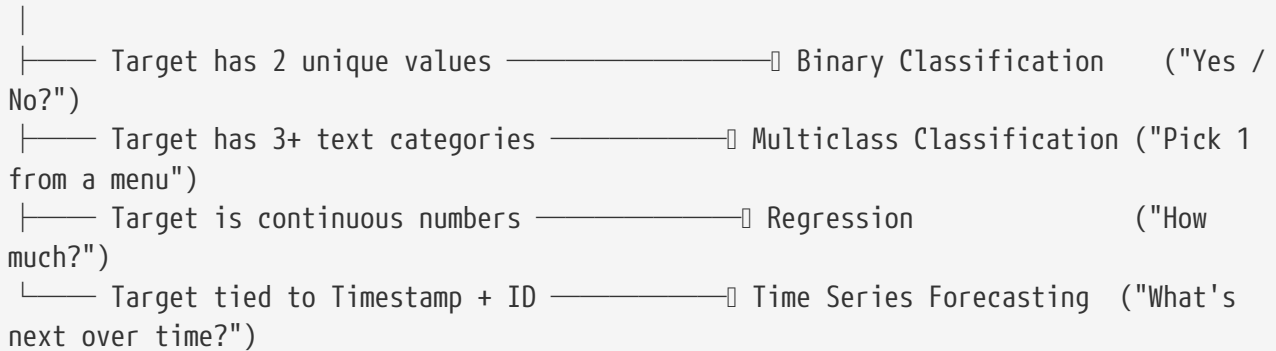
1. **What kind of value lives in the target column?** Is it a handful of labels, or a number that can slide up and down a scale?
2. **Is time part of the story?** Is each row anchored to a date or timestamp, with something identifying which sequence it belongs to?

Answer these and the task type falls out on its own. That's the skill this section builds, so that when the dashboard recommends a task type, you can confirm it rather than take it on faith.

The quick decision tree

The clue is almost always in your **target column**, the thing you're trying to predict:

Raw CSV Data



Let's take them one at a time.

1. Binary classification: "Yes or No?"

The workhorse of enterprise ML. Your target column has exactly **two** values, 0/1, True/False, Churn/No-Churn.

- **Data cue:** two unique values in the target.
- **You'll see it in:** customer churn prevention, fraud detection, credit-risk approval.
- **Default metric:** **Accuracy**, but switch to **ROC-AUC** or **F1** when the classes are lopsided. On a dataset that's 99% legitimate and 1% fraud, a model that guesses "not fraud" every time is 99% accurate and completely useless, so accuracy alone will lie to you.

2. Multiclass classification: "Pick 1 from a menu"

Same idea as binary, but now there are three or more categories to choose from.

- **Data cue:** three or more distinct text labels in the target.
- **You'll see it in:** support-ticket routing, customer segmentation, classifying **which type** of equipment failure occurred.
- **Default metric:** **Accuracy**.

3. Regression: "How much?"

When the answer isn't a category at all but a number on a continuous scale.

- **Data cue:** the target holds continuous numeric values, with no time component.
- **You'll see it in:** price estimation, predicting a patient's medical charges, estimating how long an operation will take.

- **Default metric: R^2 (R-squared)**, the share of variance the model explains, on a 0.0-to-1.0 scale. Closer to 1.0 is better.

4. Time series forecasting: "What's next over time?"

Regression's time-aware cousin. You're not just predicting a number, you're predicting it **forward**, respecting the order of events.

- **Data cue:** the dataset has a **timestamp** column, an **ID** column identifying each sequence (say, each store or product), and a numeric **target**.
- **You'll see it in:** retail inventory demand, quarterly sales projections, capacity planning.
- **Default metric: MASE (Mean Absolute Scaled Error)**, it grades the forecast against a naive "tomorrow looks like today" baseline, so you can tell whether the model is genuinely adding value or just going through the motions.

What's next

In the next chapter, we'll see the results for how AutoML actually builds and stacks these models under the covers, and how to read the evaluation "report card" before you promote a winner into production.

Validating and Deploying AutoML Models

Goal

Use AutoML evaluation artifacts (leaderboards, feature importance, ROC/PR curves) to validate business logic and promote winning models to production.

The tempting move is to grab whatever sits in the top row and ship it. Resist that for a few more minutes. A high score tells you a model fit the **training** data well, it doesn't yet tell you the model is measuring the **right thing**, or that it will act correctly in production.

We'll do it in three moves: first a quick look at what AutoML was actually doing while we waited, then how to read the full evaluation "report card" that comes back with every run, and finally the paved path from a winning model to a live, serving endpoint.

How AutoML Builds Its Models

Objective

Explain what AutoGluon is doing during a run and why the leaderboard is an ensemble, not a single algorithm.

You don't strictly **need** to know what happens between "start run" and "here's your leaderboard" to use AutoML. But a little bit of knowledge pays off the moment you start reading results, because the names you'll see on the leaderboard have meaning.

Interactive experience: click the flashing purple circle to advance the frame.

<iframe

src="https://demo.arcade.software/VzyYitxsdIfzOnXagZ2j?embed&embed_mobile=inline&embed_desktop=inline&show_copy_link=true"

width="100%"

height="600px"

frameborder="0"

allowfullscreen

webkitallowfullscreen

mozallowfullscreen

allow="clipboard-write"

muted>

</iframe>

AutoGluon

Under the covers, RHOAI AutoML is powered by **AutoGluon**, which acts as a meta-orchestrator. Instead of asking you to bet on one algorithm and tune it by hand, AutoGluon runs **parallel sweeps**

across several model families at once, then figures out which ones actually deserved their place in the line-up.

Think of an algorithm as a mathematical recipe, and a model family (or software library) as the specific engine built to execute that recipe. What AutoGluon does as a "meta-orchestrator" is manage a whole workshop full of different engines at the same time, rather than making you guess which one will work best.

Those candidates come from two broad families.

"Smart flowchart builders" (gradient-boosted decision trees)

These models learn by chaining together lots of yes/no splits, like a flowchart that keeps correcting its own mistakes. They're the tried and true of tabular prediction:

- **XGBoost**, the meticulous veteran. Uses exact, depth-wise tree splits. A dependable default for standard spreadsheet and financial data.
- **LightGBM**, the speed demon. Built by Microsoft, it uses fast histogram-based, leaf-wise splitting to chew through multi-gigabyte datasets with low memory overhead.
- **CatBoost**, the category specialist. Built by Yandex, it handles categorical text columns natively, so you don't have to manually encode them.

"Synthetic brains" (deep neural networks)

- **Tabular MLPs (PyTorch / FastAI)**, multi-layer perceptrons that capture complex, non-linear interactions between your columns, patterns the tree models can miss.

The real trick: stacked ensembling

Here's the part that matters most. AutoGluon doesn't just crown a single winning algorithm and walk away. It combines the predictions of the top candidates into a multi-layer **Weighted Ensemble**, letting the models cover for each other's blind spots. That stacked approach consistently beats hand-tuning any single model by roughly **2–5% in accuracy**.



First time seeing these names? Don't worry about choosing between XGBoost, LightGBM, and the rest. Just know that when a "WeightedEnsemble" model tops your leaderboard, it isn't a bug or a mystery entry: it's usually the blended model, and it's often your best performer.

What's next

Now, let's actually read it, along with the rest of the report card AutoML hands back with every run.

Reading the Report Card

Objective

Interpret the leaderboard and evaluation artifacts to validate a model's business logic before

promoting it.

When a run finishes, RHOAI gathers everything the pipeline produced and lays it out in the dashboard as a set of connected views. Together they're the model's **report card**. Your job here is to ask each artifact a question and make sure the answers string together before you trust the model with a real decision.

The leaderboard: who's in the running

The leaderboard is your starting point. It lists every candidate model AutoML trained, ranked by the primary optimization metric for your task (the Accuracy, R^2 , or MASE you met in chapter 1). But don't stop at the ranking column. It also shows **secondary metrics**, **fit times**, and **inference latencies**, and those extra columns often decide the real winner.



The top model isn't automatically the right one to ship. If the number-one model scores a hair higher but takes ten times longer to return a prediction, a slightly lower-ranked model with microsecond latency may be the smarter production choice. Read across the row, not just down the rank.

<iframe

```
src="https://demo.arcade.software/7NwG9wrxYCrkwoX9FttF?embed&embed_mobile=inline&embed_desktop=inline&show_copy_link=true"
width="100%"
height="600px"
frameborder="0"
allowfullscreen
webkitallowfullscreen
mozallowfullscreen
allow="clipboard-write"
muted>
</iframe>
```

Feature importance: is it learning the right thing?

This is the artifact that separates "the model scored well" from "the model is trustworthy." **Feature importance** works by shuffling each column's values and measuring how much the model's performance drops, which tells you how much the model actually leans on that input.

Use it to answer three questions:

- **Does this match reality?** If a churn model's top signals are "months since last login" and "support tickets opened," that lines up with business intuition. Good sign.
- **Is there anything suspicious?** A single feature that dominates everything else is a classic red flag for **data leakage**, where a column accidentally encodes the answer (for example, a "cancellation_date" field sneaking into a churn model). The model looks brilliant in testing and

falls apart in production.

- **Can I explain this to a stakeholder?** Feature importance is how you turn a model into a story a business owner will actually believe.



New to this? Make feature importance is the **first** thing you look at after the score, not the last. Catching a leaked column here saves you from deploying a model that's secretly cheating.

Confusion matrix: where classification goes wrong

For binary and multiclass models, the **confusion matrix** lays out actual versus predicted classes in a grid: true positives, false positives, false negatives, and true negatives. It moves you past a single accuracy number and shows you **which** mistakes the model makes.

That distinction is often a business decision in disguise. In fraud detection, a false negative (missing real fraud) usually costs far more than a false positive (flagging a legitimate charge for review). The matrix lets you see that trade-off in black and white instead of hoping the headline accuracy has it covered.

ROC and PR curves: tuning the trade-off

The **ROC (Receiver Operating Characteristic)** and **Precision-Recall** curves visualize how the model separates classes across every possible **decision threshold**, not just the default cutoff. This is where you calibrate the model to the business.

Turn the fraud model's sensitivity up and you catch more fraud but annoy more legitimate customers with reviews; turn it down and the reverse. These curves are the dial, and they let you pick the operating point that matches what your organization actually cares about.

Time series backtesting: does it hold up over time?

For forecasting models there's no single test set, because time only runs one direction. Instead, RHOAI does **rolling backtesting**: it walks the model forward through successive validation windows and charts how the forecast error behaves. You can compare predicted versus observed values in the best and worst windows, which quickly reveals whether a model is reliably useful or just got lucky in one quarter.

Putting it together

None of these views is decisive on its own. A model earns promotion when the story is consistent: a strong leaderboard metric, feature importance that matches reality with no leakage, an error profile you can live with, and, for forecasts, stable behavior across backtests. Read them together and you're validating **business logic**, not just chasing a score.

What's next

Now let's move it out of the dashboard and into production, along the paved path from a finished run to a live serving endpoint.

From Winner to Endpoint: The Production Handoff

Objective

Follow the repeatable path that promotes a validated model from a completed run to a live serving endpoint.

Once you've picked a model you trust, RHOAI gives you a short, repeatable path to get it serving real traffic, and none of it requires cluster-admin skills.

There are three exits from a completed run, and you'll use them for different reasons.

Export the notebook (for the tinkerers)

Click **Save notebook** and AutoML generates a Python Jupyter notebook with the code already written to connect to your S3 storage, load the trained predictor, and run batch scoring or your own custom validation.

Reach for this when you want to poke at the model yourself, score a big batch of records offline, or hand a data scientist a running start. It's the "show me the code" door.

```
<iframe  
  
src="https://demo.arcade.software/jwBIv65bLGvadZHbp2ms?embed&embed_mobile=inline&embed_desktop=inline&show_copy_link=true"  
width="100%"  
height="600px"  
frameborder="0"  
allowfullscreen  
webkitallowfullscreen  
mozallowfullscreen  
allow="clipboard-write"  
muted>  
</iframe>
```

Register the model (for governance)

Register promotes the winning model version straight into the **RHOAI Model Registry**. Think of the registry as the system of record for your models: it versions them, keeps their metadata, and gives your team one trusted place to answer "which model is this, and where did it come from?"



First time shipping a model? Register it even if you're eager to deploy. The registry is what makes a model **governable**, versioned, discoverable, and auditable, rather than a mystery file someone deployed one afternoon.

```
<iframe
```

```
src="https://demo.arcade.software/CjPnxZ3oa97ePxGoX958?embed&embed_mobile=inline&embed_desktop=inline&show_copy_link=true"
width="100%"
height="600px"
frameborder="0"
allowfullscreen
webkitallowfullscreen
mozallowfullscreen
allow="clipboard-write"
muted>
</iframe>
```

Deploy to a serving endpoint (for production)

This is the one that turns a file into a service. RHOAI deploys the model using the **AutoGluon ServingRuntime for KServe**, which instantly stands up a scalable REST API. Your application then gets predictions by POSTing to a standard endpoint:

```
/v2/models/<model-name>/infer
```

That's a **KServe v2** inference endpoint. Any service that can make an HTTP request, your web app, a batch job, another microservice, can now get live predictions without knowing a thing about how the model was built.

```
<iframe
```

```
src="https://demo.arcade.software/4bLBXBUANDw70srcbhfK?embed&embed_mobile=inline&embed_desktop=inline&show_copy_link=true"
width="100%"
height="600px"
frameborder="0"
allowfullscreen
webkitallowfullscreen
mozallowfullscreen
allow="clipboard-write"
muted>
</iframe>
```



If terms like KServe and "serving runtime" are new, here's the short version: KServe is the piece of OpenShift AI that runs your model as a web service and scales it for you. The AutoGluon ServingRuntime is a prebuilt engine that already knows how to load and serve AutoML models, so you don't assemble any of that plumbing yourself.

The full arc

Step back and look at the path you've traced across two chapters:

Raw CSV (S3) —> AutoML Run —> Leaderboard —> Evaluation Report Card —> Registry —> KServe REST Endpoint

Raw data went in one end, and a governed, validated, production-ready prediction service came out the other, in hours rather than months, and without you touching a cluster.

What's next

You can now take predictive, tabular data all the way from a CSV to a served model. In the next chapter we shift from numbers to **text**: an introduction to generative AI in the GenAI Playground, and using MLflow to version and govern prompts the same disciplined way you just governed models.



Looking ahead to RHOAI 3.6: AutoML is a **Technology Preview** feature in OpenShift AI 3.5. It moves to **General Availability** in 3.6 (targeted November 2026), which means the exact workflow you just learned becomes fully production-supported, same UI, same concepts, now with a production SLA behind it.

Prompt Engineering and MLflow Governance

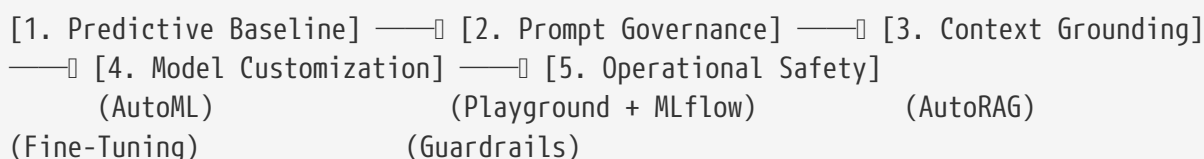
Goal

Transition from predictive data to generative text, leveraging the GenAI Studio Playground and MLflow to version, track, and eliminate "prompt drift".

This chapter is where we cross over into **text**. Same platform, same governance instinct, but now the thing you're shaping is a prompt to a large language model instead of a tabular prediction.

When you worked with AutoML, you never would have shipped a model without a leaderboard, feature importance, and a version in the registry. Yet the industry's default habit with prompts is the opposite: a clever instruction gets typed into a text box, copied into some code, tweaked by a teammate, pasted into a notebook, and six weeks later nobody can say which version is actually running in production. That sprawl has a name, **prompt drift**, and it's exactly the kind of chaos this chapter teaches you to prevent.

It helps to see where this fits in the bigger picture. Think of enterprise GenAI as an escalation ladder, and prompt governance is the second rung:



Over the next few pages we'll cover rung 2 in three moves: testing prompts interactively in the **GenAI Studio Playground**, treating those prompts as versioned **code** rather than throwaway text, and using **MLflow** to govern them across your whole organization.



Rung 5, operational safety, is a subject in its own right. The runtime input and output rails that keep a model on-policy (for example, NeMo Guardrails) are covered in the dedicated **Enabling AI Safety with Guardrails** course, so we won't detour into them here.

Testing Prompts in the GenAI Studio Playground

Objective

Use the Playground to test system instructions and generation settings, and understand what each knob actually does.

Before you version a prompt, you have to **find** a prompt worth keeping. That's what the Playground is for: a place to experiment with a language model in real time, watch how it responds, and turn

knobs until the behavior is right, all before a single line of it touches your application code.

In the RHOAI dashboard you'll find this under **GenAI Studio**. That menu item has two areas that work together: **Playground**, the interactive testing surface we cover here, and **Prompts**, the registry where good prompts go to be saved and versioned (that's the next section).

Turning the feature on (admin)

GenAI Studio and the prompt tools are gated behind two feature flags in the `OdhdashboardConfig` resource (in the `redhat-ods-applications` namespace). An administrator sets both to `true`:



```
spec:
  dashboardConfig:
    genAiStudio: true      # exposes the GenAI Studio navigation
    promptManagement: true # enables the Prompt tab and MLflow
  registry UI
```

If you enable `promptManagement` and the **Prompt** tab still doesn't appear, reload the Playground to force the dashboard to pick up the change.

From numbers to text

Quick reframe as you shift gears. With AutoML, you gave the model structured columns and it returned a structured prediction. With a language model, your "input" is **language itself**, the instructions and context you write, and the output is generated text. The quality of that output depends enormously on how you phrase the ask. So the Playground is where prompt **engineering** happens: small changes in wording and settings, big changes in behavior.

The knobs you'll actually turn

Everything you adjust lives in the **Settings** panel that opens on the right side of the Playground. Rather than a single wall of sliders, the controls are grouped into tabs, **Model**, **Prompt**, **Knowledge**, and **MCP**, plus a **Guardrails** tab, so each area of behavior has its own place. Two of these controls are wired directly to how the model generates each token; the rest shape **what** the model is working with before it ever starts writing.

Start on the **Model** tab, home to the two direct generation controls:

- **Temperature** controls randomness on a scale from **0** (focused, deterministic, repeatable) to **2** (loose and creative). This is the dial you'll reach for first when an answer feels either too rigid or too scattered.
- **Streaming** toggles real-time token rendering. Leave it on to watch a response build word by word, which is the easiest way to **feel** latency and time-to-first-token; toggle it off to receive the whole answer at once.

The Model tab is also where you pick the model itself and, if your project serves one, an **audio transcription model** (an ASR model such as Whisper) that lets you upload speech and have it

transcribed before it's sent to the chat.



If you've worked with LLM APIs, you may expect a **Top-p (nucleus sampling)** slider next to temperature. It's a real concept, a second way to constrain variety, but the Playground UI deliberately keeps **Temperature** as the single interactive sampling dial. Reach for the **View code** button (top right) when you need to set parameters the UI doesn't expose.

Move to the **Prompt** tab for the biggest lever of all:

- **System instructions** set the model's standing orders: its role, tone, and rules ("You are a concise support agent. Never invent policy numbers."). Small changes here move behavior more than any slider.
- **Template variables** let you write reusable placeholders with `{{variable_name}}` syntax. When a prompt with variables is loaded, the UI renders a text field for each one so you can fill in values before running, handy for testing the same prompt against many inputs.
- **Versioned prompts** connect this tab to the MLflow prompt registry: use **Load Prompt** to pull a saved system prompt (project-scoped or organization-wide) straight into the Playground, and save your refinements back the same way. That round trip is the heart of the next section.



First time touching these? Change **one** setting at a time. If you drop the temperature and rewrite the system instruction in the same breath, you won't know which change fixed (or broke) the output. Treat it like tuning an instrument, not flipping every switch at once.

You'll also keep an eye on the **context window**, the total amount of text (your instructions, the conversation, any supplied content) the model can consider at once. Every model has a limit, and understanding it now will matter a lot in the next chapter when we start feeding models retrieved documents.

Grounding, tools, and safety

The remaining tabs move beyond sampling and into what surrounds the model. You won't tune these on day one, but knowing they're here shapes how you think about the rest of the course:

- **Knowledge** is where RAG lives. Toggle it on to attach uploaded documents or an enterprise vector store, and use the **chunking controls** (maximum chunk length, chunk overlap, and the delimiter used to split text) to shape how those documents are indexed. This is a direct preview of the next chapter.
- **MCP** (Model Context Protocol) authorizes connected MCP servers and enables specific tools the model can call during a conversation, the bridge from a chatbot to something that can take action.
- **Guardrails** selects a dedicated safety model and lets you independently toggle **input guardrails** (screening the request before it reaches the model) and **output guardrails** (screening the response before it reaches you), so you can watch for prompt injection and unsafe content.

Comparing and tracing

The Playground isn't just a single chat box. A few capabilities make it a genuine evaluation tool:

- **A/B comparison** (the **Compare chat** button) splits the chat area into two panes so you can run two prompt versions, or two different foundation models, side by side and judge outputs directly instead of relying on memory of "what it said last time."
- **Tracing** records the call tree behind a response. Flip on **Enable tracing** and each query collects OpenTelemetry spans, so you can see what actually happened on the way to an answer, which becomes invaluable as prompts feed into larger agentic workflows.
- **Inline metrics** appear under each response, including **Time to First Token (TTFT)**, **Total Tokens**, and execution latency, with a link to open the full call tree in MLflow. These are the numbers that turn a subjective "that felt slow" into something you can actually compare.



The models you test against are the foundation models your platform serves (for example Llama, Granite, or Claude). A prompt that shines on one model may behave differently on another, which is exactly why comparing across models is built in.

What's next

Once the Playground gives you a prompt that behaves the way you want, the worst thing you can do is leave it sitting in a browser tab. Next we'll treat that prompt the way you'd treat any other production asset: as versioned code.

Prompts as Code: The MLflow Prompt Registry

Objective

Understand why prompts deserve version control, and how the MLflow Prompt Registry versions, aliases, and binds them to model settings.

A prompt is not a throwaway string, it's a **parameter** that changes your application's behavior just as surely as any line of code. Yet prompts tend to get treated like sticky notes: copied, pasted, tweaked, and scattered. When a production answer suddenly regresses, "which prompt caused this?" becomes a question nobody can answer.

That's what the **MLflow Prompt Registry** provides. In the dashboard it lives under **GenAI Studio** → **Prompts**, and if you governed a model in chapter 2, the idea will feel familiar: it's a central place to create, version, tag, and reuse prompt templates across your organization.

Versioning, the Git way

Every time you register a change to a prompt, MLflow saves a new numbered version (v1, v2, v3...) with a commit message describing what changed, just like a Git history for your instructions. Two properties make this trustworthy:

- **Versions are immutable.** Once v2 exists, it never silently changes. If you want to edit a prompt, you create v3. That guarantee is what makes a past result reproducible.
- **You can compare versions side by side**, with the differences highlighted, so you can see exactly how the wording evolved between v1 and v2.

Aliases: pointing at "the good one"

Here's the feature that keeps version numbers from leaking into your application code. An **alias** is a movable label that points at a specific version, for example a **production** alias pointing at v2, or the built-in **latest** alias that always tracks the newest version.

Your application asks for the prompt **by alias**, not by number. So when you promote v3, you just move the **production** alias to v3 and every app picks it up, no code change, no redeploy. Need to undo a bad prompt? Move the alias back. That's a one-click rollback.



New to this idea? Think of an alias like a "current" label on a shelf. The versioned bottles never change, but the label can be moved to whichever one is in service today. Your code always reaches for the label, never a specific bottle.

Because promoting an alias instantly changes what production sees, you'll want to be sure v3 is actually better before you move the label. That's where the Playground's **Compare** view earns its keep: load v2 and v3 into the two panes, run the same input against both, and judge the responses (along with time-to-first-token, total tokens, and latency) side by side. Validate first, then promote.

Prompts carry their settings with them

A prompt is more than its words. The registry can store a **model configuration** alongside each prompt: the model name, temperature, max tokens, and the other generation settings you dialed in back in the Playground. That means the recipe travels with the prompt, so a teammate loading v2 gets not just the text but the exact settings it was tuned for.

Prompts can also be **bound to a specific model provider and endpoint** (Llama, Granite, Claude, and so on). Because the application references the prompt rather than hardcoding a model, teams can swap the underlying model without rewriting prompt code.



Templates use `{{variable}}` placeholders, so a single registered prompt can be reused with different runtime values (a `{{question}}` or a `{{num_sentences}}` filled in at call time). You register the reusable **shape** of the prompt once, not a hundred near-identical copies. When you load a templated prompt into the Playground, the UI reads those placeholders and renders an input field for each one beneath the instructions box, so you fill in the runtime values before you run.

When the recipe is bigger than a prompt

The registry captures the prompt text, its versions, and the generation settings, but a serious experiment often involves more moving parts: a chosen model endpoint, attached RAG knowledge sources, and connected MCP tools. To capture that **entire** execution recipe as one reloadable unit, save the Playground session as a **Playground Agent**.

The payoff is reproducibility at the session level. When a teammate reopens the agent, they don't just inherit the prompt text and temperature, they get the exact model runtime, the same vector store, and the same tool definitions you were experimenting with, so what they see matches what you saw.

▼ *Going deeper on Playground Agents (for the curious)*

Playground Agents (also called **Saved Agents**) are a real feature in RHOAI 3.5, but they're a **Developer Preview** feature and **disabled by default**, so you won't see the option in the Playground until an admin turns it on.

Turning it on (admin). Set the `agentConfigManagement` feature flag to `true` in the `OdhDashboardConfig` resource (the `odh-dashboard-config` instance in the `redhat-ods-applications` namespace):

```
spec:
  dashboardConfig:
    agentConfigManagement: true    # exposes Save/Load agent in the Playground
```

After saving, hard-refresh the browser (**Cmd/Ctrl + Shift + R**) so the dashboard picks up the change.

What a saved agent captures: your model selection, inference parameters (such as temperature), MLflow prompt references, RAG knowledge sources (uploaded documents or vector stores), and MCP server connections.

What it deliberately does not capture: chat history, guardrail settings, or secret credential values. A saved agent is the **recipe**, not the conversation or the secrets.

Scope: saved agents are stored as Kubernetes resources in the project namespace, so everyone with access to the project can view, load, and collaborate on them, the same isolation model as the prompt registry itself.

Using them:

- **Save** → configure the Playground, then use the header menu → **Save agent**, give it a name and optional description.
- **Load** → header menu → **Load agent**, or go to **GenAI Studio** → **AI asset endpoints** → **Agents** tab and click **Try in Playground**.
- **Make a variant** → load an agent, change a setting (temperature, prompt version...), then **Save as new agent** to branch a copy without overwriting the baseline.
- **Manage** → rename, edit descriptions, or delete agents from the **AI asset endpoints** → **Agents** tab.

See it in action

The end-to-end registration flow, creating a prompt, committing a v2, comparing versions, and loading one back into the Playground, is best seen rather than read.

A short walkthrough demo accompanies this section and steps through registering and versioning a prompt in the UI.

What's next

You can now version a single prompt like code. The last piece is scale: how these registries are isolated per project, shared across the enterprise, and tied into a complete audit trail. That's governance, and it's next.

Governance at Scale: Workspaces, Lineage, and Audit

Objective

Explain how MLflow workspaces isolate and share prompts, and why the resulting audit lineage matters to the enterprise.

Versioning a single prompt is a personal win. Governance is the **team** win: making sure the right people can reach the right prompts, that nothing important is ever untracked, and that when someone in risk or compliance asks "what generated this answer six months ago?", you have a real answer.

Workspaces: one per project

MLflow organizes work into **workspaces**, and RHOAI maps them cleanly to something you already know: an OpenShift project namespace corresponds **1:1** to an MLflow workspace. Your project's experiments, runs, and registered prompts all live inside that boundary, and access is controlled by the same Kubernetes RBAC that governs the rest of the platform.



If "namespace" and "RBAC" are new terms, the short version is: a namespace is a walled-off area for one project's work, and RBAC is the rulebook for who's allowed in. You don't set this up yourself as a practitioner, you inherit the isolation your project already has.

Global registries: shared, vetted prompts

Isolation is great for day-to-day work, but some prompts deserve to be shared across the whole company, think a security-vetted system instruction that every team should build on. For that, a cluster administrator can designate certain namespaces as **global** prompt registries, granting other teams either read-only or edit access.

The result is a central, trusted library of enterprise prompts: individual teams iterate in their own workspaces, while the organization maintains a shared shelf of approved instructions nobody has to reinvent.



Designating a global registry (admin)

Setting up a shared registry takes three steps, all performed by a cluster administrator:

1. Add the namespace to `spec.globalMLflowNamespaces` in the `OdhDashboardConfig` (this field sits directly under `spec`, **not** under `spec.dashboardConfig`), or select it under **Settings** → **General settings** → **Global project** in the dashboard.
2. Label the namespace so the platform treats it as a shared workspace:

```
oc label namespace global-prompt-workspace opendatahub.io/global-mlflow-workspace=mlflow
```

3. Make sure an MLflow instance is running in that namespace. The operator only creates the shared `odh-group-mlflow-view` role bindings once an active MLflow server pod is present.

Once that's in place, other teams see these prompts read-only and can use **Save As** to copy one into their own project before editing.

Complete audit lineage

This is the payoff that makes governance more than bookkeeping. For every inference, MLflow can log the exact prompt string, the input variables, the model ID, the generation settings, any evaluation scores, and the output that was produced. Nothing floats free.

That lineage buys you three things practitioners and their organizations genuinely need:

- **Reproducibility.** You can recreate any past result because you know precisely what produced it.
- **Rollback.** When a production response regresses, you trace it to a specific prompt version and revert, exactly the alias move from the last section, now with the evidence to justify it.
- **Compliance.** Risk and audit teams can prove which prompt, model, and parameters generated any historical response, which is often a hard requirement in regulated industries.

One platform for models and prompts

Here's a detail that's easy to undervalue: this is the **same** MLflow that tracked your AutoML accuracy scores in chapter 2. The instance versioning your predictive models also versions your prompt templates and traces your LLM calls. Your team doesn't juggle one tool for predictive ML and a separate one for GenAI, it's one governed platform across both, which is a big part of why the discipline actually sticks.

Where this leaves you

Step back and look at what you can now do. You can take raw tabular data all the way to a served predictive model (chapters 1 and 2), and you can take a raw idea for a prompt all the way to a versioned, governed, auditable instruction bound to a model (this chapter). Both halves of "connecting data to models" are now under proper control.

But a well-governed prompt still only knows what the model already knows. The moment you need it grounded in **your** proprietary documents, prompt engineering hits a ceiling. That's rung 3 of the

ladder, and it's where the next chapter picks up: **AutoRAG**, which automates the retrieval pipeline that feeds your governed prompts the right context.

Generative Retrieval with AutoRAG

Goal

Position AutoRAG as an automated optimization engine that benchmarks chunking, embedding, and vector database configurations for grounded LLM applications.

At the end of chapter 3 we hit an honest limit: a beautifully governed prompt still only knows what the model already learned during training. Ask it about **your** company's return policy, last quarter's field procedures, or the fine print in a specific contract, and it will either shrug or, worse, confidently make something up.

The standard fix is **RAG**, Retrieval-Augmented Generation: before the model answers, you retrieve the relevant chunks from your own documents and hand them over as context. It works. The catch is that building RAG **well** is a grind. How big should the chunks be? Recursive or structural chunking? Vector search or hybrid? Which embedding model? How many chunks to retrieve? Every combination is a knob, and tuning them by hand is weeks of trial and error, exactly the kind of manual benchmarking you learned to avoid back in chapter 1.

AutoRAG is AutoML's idea applied to retrieval. Instead of hand-tuning, you give it your documents and a set of test questions, and it automatically searches across chunking, embedding, and retrieval configurations, scores each one against your questions, and hands back a ranked leaderboard of production-ready RAG patterns. If chapter 1's AutoML was a Sorting Hat for models, AutoRAG is a Sorting Hat for retrieval pipelines.



AutoRAG is a **Technology Preview** feature in OpenShift AI 3.5. It's the platform's rung-3 answer, **context grounding**, right after prompt governance and before you'd ever consider fine-tuning a model.

The demo below walks the whole thing end to end for a fictitious bank. It's worth watching once now for the shape of it; we'll unpack each stage in the pages that follow.

▶ [autorag-demo.mp4](#) (video)

AutoRAG Demo - OpenShift AI 3.5

We'll cover it in three moves: preparing the two inputs AutoRAG needs, understanding the search space it explores and how to read the metrics it reports back, and finally inspecting the winner and exporting it into a real application.

Preparing the Inputs: Documents and a Test Set

Objective

Assemble the two inputs an AutoRAG run needs: a document corpus and a ground-truth evaluation dataset.

An AutoRAG run is only as good as what you feed it, and it needs exactly two things: the **documents** you want to answer questions about, and a **test set** that defines what a good answer looks like. Get these right and the rest of the run mostly takes care of itself.

Input 1: your document corpus

This is the knowledge you want the model grounded in. A few practical rules:

- **Supported formats:** PDF, DOCX, PPTX, Markdown, HTML, and TXT.
- **Where it lives:** an S3-compatible bucket, or uploaded directly through the UI.
- **Size:** up to 32 MiB per file when you upload through the UI; no per-file limit when you point AutoRAG at an S3 folder. Either way, a run automatically **samples up to 1 GiB** of documents.



First time out? Start small and clean. A handful of high-quality, text-based documents will teach you more about how AutoRAG behaves than dumping in a giant messy archive. You can scale up once you trust the workflow.



In Technology Preview, AutoRAG does not run OCR on image-only PDFs or pull text out of embedded images. If your PDFs are scanned pages, the text inside them won't be retrievable. Table structure extraction **is** available, but only when you choose the **Better Quality** run preset.

Input 2: the evaluation dataset (the answer key)

This is the piece newcomers tend to underestimate, and it's the most important. AutoRAG can't rank retrieval patterns unless it knows what "correct" means, so you give it an **answer key**: a JSON file of representative questions paired with their known-good answers and the documents those answers come from.

Each entry has three fields:

```
[
  {
    "question": "What is the return policy?",
    "correct_answers": [
      "Items can be returned within 30 days",
      "The return window is 30 days from purchase"
    ],
    "correct_answer_document_ids": ["policies.pdf", "faq.md"]
  }
]
```

- **question** is a real question a user might ask.
- **correct_answers** is a list, because the same answer can be phrased many ways. Give a few variations so a good answer isn't marked wrong on wording alone.
- **correct_answer_document_ids** names the source documents that actually contain the answer. This

is what lets AutoRAG score **retrieval**, not just the final text.



`correct_answer_document_ids` must use **base filenames only** (for example `policies.pdf`), never a folder path like `documents/policies.pdf`. This trips up a lot of first runs.

Building it without the busywork

In earlier releases you had to download a template, fill it in offline, and re-upload it. OpenShift AI 3.5 adds an **inline evaluation-file creator**: you can type a question, its answer, and the source document straight into the UI (you'll see this in the demo). For a couple of test cases that's the quickest path; for a serious evaluation set you'll still want to prepare a fuller JSON file with many questions covering the range of things users will actually ask.

What's next

With your documents and your answer key in place, AutoRAG has everything it needs to start experimenting. Next we'll look at the **search space** it explores on your behalf, and, just as importantly, how to read the scores it reports back.

The Search Space and the Metric Triad

Objective

Understand the configurations AutoRAG searches and how to interpret the three metrics it scores them on.

This is the heart of AutoRAG. Once you kick off a run, it systematically tries combinations of retrieval settings, scores each against your evaluation set, and ranks them. Two things are worth understanding here: **what** it's varying, and **how to read** the scores that come back.

What AutoRAG searches

Think of every setting below as a dial. Tuning them by hand means testing one combination, waiting, adjusting, and repeating for weeks. AutoRAG sweeps them for you:

- **Chunking strategy**, how documents are split. Recursive chunking versus hybrid structural chunking (the latter preserves headings and table structure, and kicks in with the **Better Quality** preset).
- **Chunk size and overlap**, how big each piece is and how much neighboring pieces share.
- **Search mode**, pure vector (semantic) search versus hybrid search that blends semantic and keyword matching.
- **Top-k**, how many chunks to retrieve for each question.
- **Embedding model**, the model that turns text into vectors. You can select up to **2 embedding models** per run.

- **Foundation model**, the LLM that writes the final answer. Up to **3 foundation models** per run.



Stay within **3 foundation models and 2 embedding models** per run. Going over that cap causes the run to fail, so it's a limit to respect, not push.



The recommended default embedding model is **BAAI/bge-m3**. It supports over 100 languages, does both dense and sparse retrieval, and runs with low memory overhead, which is why it shows up throughout the AutoRAG material.

A quick word on languages

AutoRAG automatically **detects the language** of your evaluation questions, so you don't declare it up front, and it handles mixed-language corpora gracefully. One caveat worth remembering: auto-detection only helps if the **models** you picked actually support those languages. Check the Model Catalog to confirm your chosen foundation and embedding models cover your document languages.

Reading the metric triad

AutoRAG scores every candidate on three complementary metrics. You met these by name in the course overview; here's what each one actually tells you:

- **Answer Faithfulness**, a hallucination check. Did the answer stick to the retrieved context, or did the model make things up?
- **Answer Correctness**, a ground-truth check. Did the answer match the known-good answers in your evaluation set?
- **Context Correctness**, a retrieval check. Did the system fetch the **right** document chunks in the first place?

The reason there are three, not one, is that they isolate **where** a pattern succeeds or fails. And the real skill, the AutoRAG equivalent of reading a confusion matrix in chapter 2, is interpreting them **in combination**.

Diagnosing the combinations

This little decision guide is the most practical thing on the page. When two metrics disagree, the pattern is telling you something specific:

- **High Faithfulness + Low Correctness**: the model answered faithfully from what it retrieved, but the retrieved context didn't actually contain the true answer. Suspect your **chunking** or a mismatch in your **evaluation dataset**.
- **High Correctness + Low Faithfulness**: the answer was right, but the model pulled it from its own training memory rather than your documents. It got lucky this time; on unseen questions it's a **hallucination risk**.
- **Low Context Correctness + High Answer Correctness**: retrieval fetched irrelevant chunks, but a strong foundation model compensated. It looks fine now but is **fragile**, it will break on query types it hasn't effectively memorized.



New to evaluating RAG? Don't chase a single top score. A pattern where all three metrics are solid is far more trustworthy in production than one with a dazzling Answer Correctness propped up by weak retrieval. Consistency beats a lucky headline number, the same lesson the AutoML report card taught.

What's next

You can now read the leaderboard and understand **why** a pattern ranks where it does. The last step is turning a winning pattern into something your application can actually call, which is where inspection and export come in.

Inspecting the Winner and Shipping It

Objective

Inspect a completed run with the visual pipeline and Playground, then export the chosen pattern into an application.

A ranked leaderboard is a great place to be, but you still shouldn't ship a pattern you've only seen as a number. AutoRAG gives you two ways to **inspect** a run before you commit, and two ways to **export** the pattern you choose.

Inspect: the visual pipeline (DAG)

Earlier releases drew an AutoRAG run as a single opaque line, if a step failed, good luck figuring out where. OpenShift AI 3.5 replaces that with a full **visual pipeline**, a diagram of every stage in the run.

When everything succeeds, it's a satisfying green path. When something fails, it's genuinely useful: you can see **exactly which step** broke, when, and what it reported, instead of staring at a generic error. For a first-time user, this is the difference between debugging and guessing.

Inspect: "Try this pattern" in the Playground

This is the feature people appreciate most, because it removes a classic RAG frustration: exporting a pattern, wiring it up somewhere else, testing it, and only **then** discovering it isn't what you wanted.

From the leaderboard, select **Try this pattern** on any ranked configuration to open an interactive chat panel, grounded in your document vector store, right inside OpenShift AI. Ask it real questions and watch how it answers. You can also see **which chunks** it retrieved and where they came from, so you're not just judging the answer, you're judging the **evidence** behind it. You can switch between candidate patterns live to compare them head to head.



Before exporting anything, throw a few of your own tricky questions at the pattern here. If it retrieves sensible chunks and grounds its answers in them, you've validated the pattern the cheap way, no code required.

Export: Responses API code snippets

When you're happy with a pattern, click **View code** to get an application-ready snippet that calls the OGX Responses API endpoint. You get your choice of language:

- cURL
- Python
- Go
- Node.js

These call the `/v1/responses` endpoint directly, so your application can start getting grounded answers without you writing the retrieval plumbing yourself.

Export: Jupyter notebooks

If you'd rather work hands-on, AutoRAG also generates two self-contained Jupyter notebooks: an **indexing** notebook and an **inference** notebook. Run them inside an OpenShift AI workbench to reproduce, audit, or batch-score the pattern locally, the same "here's the code" escape hatch AutoML offered for predictive models.



Governance still applies here. The system instruction you settle on for your RAG app can be versioned in the MLflow Prompt Registry from chapter 3, so a production-grade prompt and a production-grade retrieval pattern are both under proper control.

What's next

That completes the AutoRAG story, and with it, the full arc of the course's capabilities: predictive models with AutoML, governed prompts with MLflow, and grounded generation with AutoRAG. In the final chapter we'll bring all three together into one end-to-end workflow and check your understanding with a graded assessment.



Looking ahead to RHOAI 3.6: like AutoML, AutoRAG is a **Technology Preview** feature in 3.5 and moves to **General Availability** in 3.6 (targeted November 2026). The headline 3.6 addition is **DSPy-based prompt pre-optimization**: rather than hand-writing the system instruction, AutoRAG can automatically generate and score prompt candidates against a sample of your evaluation questions, then feed the winners into the same search space you tuned on this page. Think of it as automated prompt engineering layered on top of automated retrieval tuning, the two-stage version of what you just did by hand.

Knowledge Review and Assessment

Goal

Track core enablement concepts across AutoML, MLflow prompt tracking, and AutoRAG to validate practitioner capability in diagnosing enterprise data scenarios and selecting optimal AI patterns.

We've covered a lot of ground. Back in chapter 1 you had a raw CSV sitting in a bucket and a question you couldn't answer by hand. By chapter 2 that data was a ranked leaderboard and a deployed model behind a REST endpoint. Chapter 3 moved from numbers to text and put your prompts under version control so they'd stop drifting. Chapter 4 grounded those prompts in your own documents with AutoRAG. Four capabilities, one platform.

Here's the thing though: in a real enterprise, you rarely use just one of them. The interesting work happens when they combine, when a predictive model and a grounded generator sit in the same workflow and hand data back and forth. That's what this final chapter is about. First we'll walk one realistic end-to-end scenario so you can see the pieces click together, then we'll sharpen the decision logic you'll actually be graded on: which capability fits which problem, how to read the metrics when something looks off, and, just as important, when "Auto" is enough and when it isn't.

After that comes the graded assessment. It's scenario-based on purpose. Nobody's going to quiz you on trivia; the questions put you in the seat of someone diagnosing a data problem and asking "which pattern do I reach for, and how do I know it's working?"

We'll do it in two moves: a synthesis section that connects everything into a single workflow and hands you a one-page decision cheat sheet, then the quiz itself.

Connecting Data to Models: The End-to-End Workflow

Objective

Combine AutoML, MLflow, and AutoRAG into a single decision workflow and reason about when automation alone is sufficient.

Individually, each capability in this course solves one problem. Together they solve a business process. The clearest way to see that is to follow a single decision from raw data all the way to a human making a call, so let's walk a loan desk.

The end-to-end workflow: a loan desk

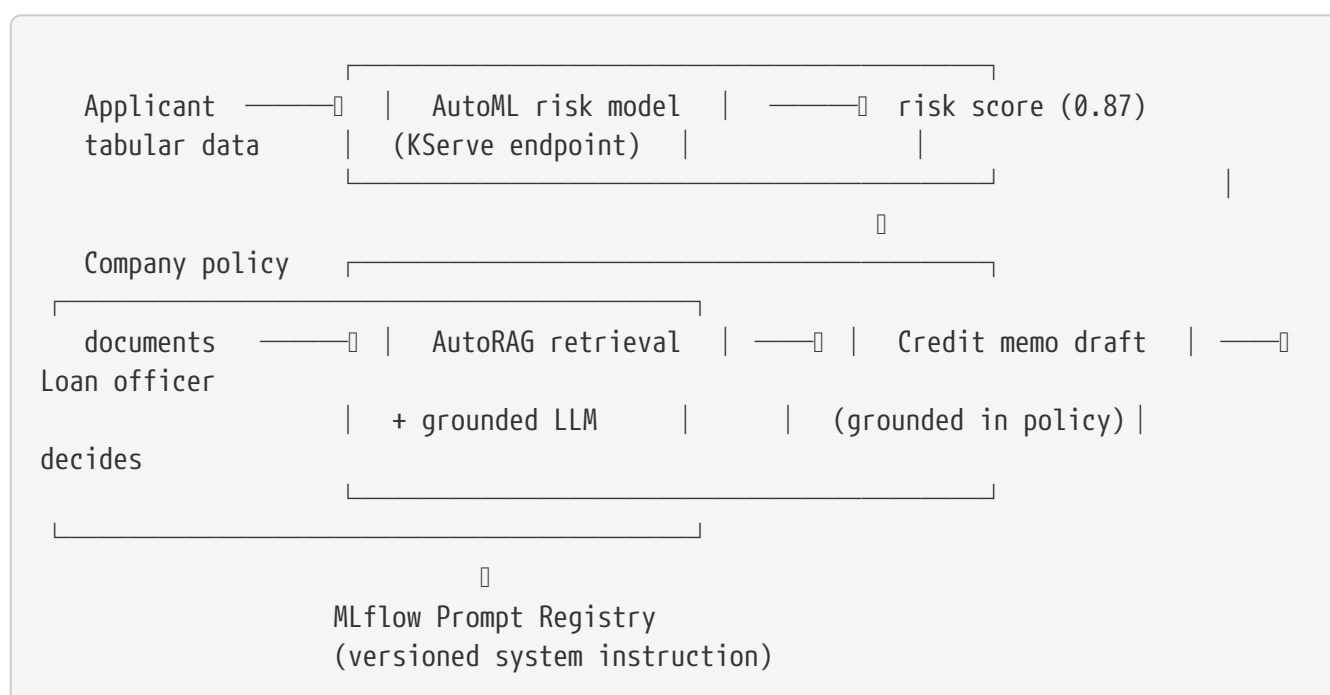
Picture a loan officer reviewing an application. Two very different questions have to be answered before they decide, and each maps to a capability you already know.

Question one: how risky is this applicant? That's a prediction over structured, tabular data, credit score, income, loan amount, history. This is textbook **AutoML**. You feed the historical loan data in, AutoML sorts through XGBoost, LightGBM, CatBoost, and neural nets, and hands back a

ranked leaderboard. You promote the winner to an **AutoGluon ServingRuntime** on KServe, and now a risk score comes back from a REST call in microseconds. That's chapters 1 and 2, start to finish.

Question two: why that score, and what does our policy say to do about it? A number alone won't satisfy a loan officer or a compliance reviewer. Someone needs a written credit memo: "This applicant scored in the elevated-risk band; per section 4 of our lending policy, elevated-risk applications above \$250k require a secondary review." That's unstructured text grounded in **your** documents, which is exactly **AutoRAG**. It retrieves the relevant policy chunks and drafts the memo, grounded in real company policy rather than whatever the base model happened to memorize.

And underneath both, quietly, is **MLflow**. The system instruction that shapes that credit memo ("write in a neutral compliance tone, cite the policy section, never invent a policy number") is a versioned prompt in the registry from chapter 3. When audit season comes, you can prove exactly which prompt version wrote which memo.



Notice what the human still does: they **decide**. AutoML gave them a fast, defensible score. AutoRAG gave them a grounded explanation and the relevant policy in plain language. The platform did the tedious parts so the officer could spend their judgment where it matters. That's the whole "Connecting Data to Models" thesis in one picture: predictive AI for the numbers, generative AI for the language, governance holding both accountable.

Diagnostic decision logic

Building the workflow is one skill. Knowing what to do when a metric looks wrong is the other, and it's the one that separates someone who **ran** AutoML and AutoRAG from someone who can **operate** them.

You already have both halves of this. From chapter 2, the AutoML report card: don't trust Accuracy on imbalanced data, read the confusion matrix and ROC/PR curves, and check Feature Importance to make sure the model isn't leaning on something it shouldn't. From chapter 4, the AutoRAG metric triad and its combinations. The single most useful diagnostic to keep in your head:

- **High Faithfulness + Low Correctness** means the model answered honestly from what it retrieved, but retrieval handed it the wrong context. The fix lives in **retrieval**, chunking or the evaluation set, not in the model.
- **High Correctness + Low Faithfulness** means the answer was right but the model pulled it from memory, not your documents. It got lucky, and luck doesn't survive contact with unseen questions.

The habit worth building: when a result looks off, ask **which stage** is failing before you start changing knobs. A low score is a symptom; the metric combination is the diagnosis.

When "Auto" is enough

Here's a question the marketing rarely answers honestly: when do you **stop**? Automation is powerful, but it isn't the answer to every problem, and knowing its edges is a mark of competence, not doubt.

Stay with the automated capabilities when:

- Your problem is predictable from structured, tabular data. **AutoML** almost certainly beats a hand-built baseline, and faster.
- Your GenAI app needs **your** facts, and a good grounded answer is enough. **AutoRAG** plus a governed prompt handles a huge share of enterprise use cases without touching model weights.
- The base model already has the **skill** you need (summarizing, extracting, answering) and just lacks your **knowledge**. Grounding solves a knowledge gap; it can't teach a skill.

Consider escalating beyond automation when:

- The model needs to reliably **behave** a certain way, a specialized tone, a strict output format, a domain vocabulary it keeps getting wrong, that no amount of prompting or retrieved context fixes. That's a **skill** gap, and it points toward fine-tuning a smaller, specialized model.
- You've genuinely exhausted prompt governance and retrieval quality and the failures persist.



That escalation, fine-tuning small language models for behavior you can't prompt or retrieve your way to, is the subject of **Course 2** in this series. The goal of **this** course is to make sure you don't reach for that heavier, more expensive tool until you actually need it. Most of the time, "Auto" really is enough.

The cheat sheet: Connecting Data to Models

Keep this near your desk. It's the one-page distillation of the whole course: match the data to the capability, then match the symptom to the fix. There's a printable one-pager you can download and keep, and the tables below preview what's on it.

[Download the one-page cheat sheet \(PDF\)](_attachments/connecting-data-to-models-cheat-sheet.pdf)

Table 1. Which capability fits the problem?

If your input is...	And you need to...	Reach for...
Structured / tabular data (CSV, database)	Predict a Yes/No, a category, a number, or a future trend	AutoML (binary, multiclass, regression, or time-series task)
A prompt that keeps drifting across app versions	Version, track, and audit prompts and generation settings	MLflow Prompt Registry (chapter 3)
A GenAI app that must answer from your documents	Ground answers in a private corpus without training a model	AutoRAG (retrieval pattern + Responses API)
A model that won't behave right no matter the prompt or context	Change the model's skill, tone, or format	SLM fine-tuning (Course 2), the escalation beyond "Auto"

Table 2. Pick the AutoML task type

The target is...	Task type	Default metric
Two outcomes (Fraud / Not fraud)	Binary classification	Accuracy (watch it on imbalanced data)
Three or more categories (a "menu")	Multiclass classification	Accuracy
A continuous number ("how much?")	Regression	R ²
A number over time ("next quarter")	Time-series forecasting	MASE

Table 3. Diagnose the AutoRAG metric triad

The pattern shows...	Which means...	Look at...
High Faithfulness + Low Correctness	Answered honestly, but retrieval fetched the wrong context	Chunking strategy, evaluation dataset
High Correctness + Low Faithfulness	Right answer, but from model memory, not your docs (hallucination risk)	Grounding; distrust it on unseen questions
Low Context Correctness + High Answer Correctness	A strong model masked weak retrieval (fragile)	Retrieval quality before it breaks in production
All three solid and balanced	Trustworthy pattern	Ship it (consistency beats a lucky headline score)



The downloadable PDF above expands each of these into a fuller reference, including business-problem cues, key decision trade-offs, the generated artifacts for every capability, and the full diagnostic-to-remediation matrix. Grab it and keep it handy.

Where this is heading: RHOAI 3.6

One forward-looking note as you finish. Everything you've built in this course runs on capabilities that are **Technology Preview** in OpenShift AI 3.5. Both AutoML and AutoRAG move to **General Availability** in 3.6 (targeted November 2026), so these workflows become fully production-supported with a proper SLA. AutoRAG also gains **DSPy-based prompt pre-optimization**, which automatically tunes the prompts feeding your retrieval pattern before the search space ever runs. It's the same "let the platform do the tuning" philosophy you've seen throughout, now applied to prompt engineering itself.

What's Next

With the full workflow and decision logic in view, the final section is a scenario-based graded assessment that validates your ability to diagnose enterprise data scenarios and select the optimal AI pattern.

Quiz: Diagnosing Scenarios and Selecting AI Patterns

Objective

Validate practitioner capability by diagnosing enterprise data scenarios and selecting the optimal AI pattern across AutoML, MLflow, and AutoRAG.

Interactive Assessment

Time to put it together. The 15 questions below are scenario-based: each one drops you into a situation, a CIO asking why AutoML matters, a banking team picking a task type, a RAG run with a suspicious metric combination, and asks you to make the right call. Pick an answer to see immediately whether it's right and **why**; the explanations are part of the learning, so read them even when you're sure.

You need **80% or higher** to pass. If you don't clear it the first time, revisit the relevant chapter and the cheat sheet in the previous section, then restart. There's no penalty for trying again.

```
<iframe type="text/javascript" src='_attachments/quiz.html?v=1' style="width: 100%; height: 850px; border: 0;" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
```

What's Next

Congratulations, you have completed **Governing AI Data Workflows: AutoML and AutoRAG Strategies**. Welcome to the world of strategically evaluating use cases and applying AutoML and AutoRAG in Red Hat OpenShift AI 3.5+ to drive measurable business value. This course is the first entry in the "Connecting Data to Models and Agents" series; Course 2 continues into SLM fine-tuning for scenarios where automation alone is not enough.